

Trabajo Practico Integrador Grupo 7

Informe Completo Con todas las Fases

Integrantes del Grupo 7:

Lombardi Lucas

Dominguez Lucas

Cuenca Enzo

Gabutti Gabriel Gustavo

FASE 1: EL PROBLEMA DE LOS 3 FOCOS

1.1. Fundamentos del Diseño Funcional e Inmutabilidad Absoluta

El desarrollo del núcleo lógico para el Sistema de Semáforos Inteligentes se estructuró bajo las pautas estrictas del paradigma funcional puro, rompiendo de forma categórica con los esquemas de la programación imperativa tradicional. En el diseño implementado, se erradicó por completo el uso de variables globales mutables (`defparameter`, `defvar`) y operadores de asignación destructiva que provoquen efectos colaterales en la memoria (`setq`, `setf`). Para modelar un entorno físico intrínsecamente cambiante como un semáforo sin alterar el estado de la memoria, se recurrió a la representación algebraica de los estados mediante símbolos literales inmutables: `'en-rojo`, `'en-verde`, `'en-amarillo` y `'amarillo-intermitente`.

El paso del tiempo y las transiciones secuenciales no modifican una celda de memoria existente; en su lugar, el flujo de datos y el estado actual del tráfico y del tiempo viajan de manera explícita y exclusiva a través de los argumentos de las funciones, respetando la secuencia vial real de seguridad (salto directo de rojo a verde, y uso de la fase amarilla estrictamente como advertencia preventiva de frenado posterior al verde). Cada ciclo del sistema evalúa las precondiciones de entrada y computa una nueva estructura de datos de salida totalmente aislada, garantizando la predictibilidad matemática y la consistencia referencial del software

. 1.2. Análisis Taxonómico de las Funciones Núcleo

A continuación, se detalla la clasificación taxonómica obligatoria de las funciones desarrolladas en Common Lisp para dar cumplimiento a los requerimientos 1 al 7 y la extensión de intermitencia de la iteración 2:

I. Función transición

- Naturaleza: Pura. Dado un color actual y una orden de cambio equivalentes, el resultado devuelto es determinista y no genera alteraciones colaterales en el entorno de ejecución.
- Estrategia de Control: Función Predicado / Condicional Declarativo. Evalúa mediante cláusulas lógicas con el macro cond la validez de la secuencia de tráfico vehicular.
- Impacto en Memoria: No destructiva. Instancia y retorna una nueva estructura de lista en cada invocación empleando la primitiva list. El diseño computa y proyecta en el primer elemento de la lista el símbolo del nuevo estado resultante inmutable (v.g., 'en-verde) en lugar del estado de origen, garantizando la correcta transición de fases exigida en la especificación de la cátedra sin alterar los punteros de los argumentos de entrada.

II. Función timer

- Naturaleza: Pura. Recibe de forma determinista una marca de tiempo Unix (Epoch) y mapea matemáticamente el color activo correspondiente al período relativo.
- Estrategia de Control: Composición funcional y aritmética plana. Emplea una operación de aritmética de módulo sobre el tiempo acumulado del ciclo total para deducir la fase actual.
- Impacto en Memoria: No destructiva. Genera un símbolo como retorno independiente, preservando intacto el valor entero del timestamp ingresado.

III. Función registrar-cambio

- Naturaleza: Impura. Su propósito explícito es interactuar con el subsistema de Entrada/Salida (I/O) del entorno, generando un efecto secundario visible al escribir cadenas formateadas en la consola de auditoría forense.
- Estrategia de Control: Función de flujo secuencial / Mapeo de salida.
- Impacto en Memoria: No destructiva. No altera estructuras de datos del programa; su salida se consume directamente en el stream de la terminal.

IV. Funciones de Análisis de Ciclos (duración-ciclo y recomendacion-ciclo)

- Naturaleza: Pura. Realizan cálculos y evaluaciones estrictamente deterministas sobre rangos estándar de ingeniería de tráfico.
- Estrategia de Control: Condicional Declarativo.
- Impacto en Memoria: No destructiva. Retornan constantes numéricas o cadenas de texto literales sin modificar parámetros previos.

1.3. Estrategias de Optimización y Simulación del Tiempo (TCO)

Para ejecutar la simulación del paso del tiempo y dar cumplimiento al Requerimiento 7 (Aseguramiento de la Calidad) sin recurrir a estructuras de control imperativas prohibidas (while, loop), se diseñó la función motora simular-linea-tiempo aplicando Recursividad de Cola (Tail Recursion).

En un esquema de recursividad simple, cada llamada descendente retiene un contexto operativo en la pila de ejecución esperando resolver una operación posterior (por ejemplo, acumular o evaluar un dato rezagado), lo que genera un crecimiento lineal de la memoria de

tamaño $O(n)$. Frente a simulaciones de tiempos prolongados, este comportamiento agota los recursos del entorno y gatilla un desborde crítico.

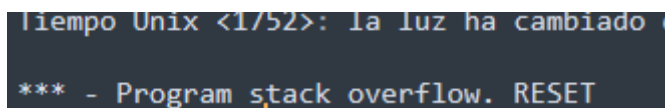
Nuestra solución sitúa la llamada recursiva a simular-linea-tiempo como la última expresión absoluta a ser evaluada en el cuerpo de la función. Esto permite que el compilador de Common Lisp aplique de forma nativa la Optimización de Llamada de Cola (TCO, Tail Call Optimization), transformando internamente la cascada de llamadas en un salto iterativo a bajo nivel. Como consecuencia, el marco de la pila (stack frame) se reutiliza indefinidamente, reduciendo el consumo de espacio en memoria a una constante $O(1)$ y permitiendo procesar flujos de tiempo extensos con total seguridad y estabilidad en el sistema embebido.

1.4. Bitácora de Depuración y Errores Conceptuales

Durante la transición hacia el pensamiento funcional, el equipo experimentó y resolvió los siguientes cinco fallos de carácter conceptual:

Bug 1: Control Stack Exhausted por Recursión Ineficiente

- **Síntoma:** Al ejecutar pruebas unitarias con marcas de tiempo representativas de varias horas de operación, el entorno de desarrollo (SBCL) abortaba abruptamente la ejecución arrojando la excepción: Control stack exhausted.
- **Causa Conceptual:** El prototipo inicial de la función de simulación incrementaba el tiempo sumando un valor de manera diferida fuera de la llamada recursiva, impidiendo la liberación de la pila. El programa acumulaba cada segundo transcurrido como un marco pendiente en memoria.
- **Diagnóstico y Corrección:** Se reestructuró la lógica eliminando cualquier cálculo diferido. Los parámetros incrementales (tales como $(+ \text{timestamp-actual } 1)$) se inyectaron directamente dentro del paso de argumentos de la función recursiva, garantizando una llamada de cola pura optimizable por el compilador mediante la reutilización del marco operativo.



```
Tiempo Unix <1752>: la luz ha cambiado o
*** - Program stack overflow. RESET
```

Bug 2: Evaluación Errónea de Símbolos (Unbound Variable)

- **Síntoma:** Al ingresar pruebas manuales en el REPL como (transicion en-rojo verde), el evaluador interrumpía el hilo levantando una excepción del tipo: Cell-error: The variable EN-ROJO is unbound.
- **Causa Conceptual:** Confusión conceptual entre un símbolo interpretado como dato y un símbolo evaluado como variable. El lector de Lisp intentaba buscar el valor en memoria apuntado por la palabra en-rojo en lugar de tratarla como un átomo identificador único en sí mismo.
- **Diagnóstico y Corrección:** Se aplicó el mecanismo de cotización anteponiendo una comilla simple ('en-rojo). Esto le instruye explícitamente al entorno que suspenda la evaluación regular

de variables y procese el símbolo como un valor literal inmutable para las comparaciones de estados del semáforo.

```
*** - AND: variable EN-ROJO has no value
Es posible continuar en los siguientes puntos:
USE-VALUE      :R1      Input a value to be used instead of EN-ROJO.
```

Bug 3: Acoplamiento de Efectos Secundarios en Funciones Matemáticas

- **Síntoma:** Dificultad extrema para realizar pruebas unitarias automatizadas sobre la función `timer`, ya que cada cálculo matemático provocaba impresiones de logs desordenadas y asíncronas en la pantalla de depuración.
- **Causa Conceptual:** Se corrompió la pureza referencial al mezclar lógica de procesamiento matemático (el cálculo del color según el residuo del tiempo Unix) con acciones imperativas de Entrada/Salida (el uso del macro `format` para interactuar con el flujo de la consola).
- **Diagnóstico y Corrección:** Se aplicó el principio de separación estricta de responsabilidades funcionales. Se depuró la función `timer` removiendo toda lógica de impresión para clasificarla como una función puramente matemática y determinista. El mecanismo de logging de auditoría forense fue aislado por completo dentro de la función específica impura `registrar-cambio`.

Bug 4: Fallo de Compilación por Dependencia Inexistente del Sistema de Paquetes (Quicklisp)

- **Síntoma:** Al intentar evaluar o compilar el archivo de lógica unificada (`core.lisp`) mediante la primitiva `load` en el entorno interactivo (REPL), el compilador interrumpía la ejecución abriendo el lazo de depuración activa (debugger) con la excepción: `READ error: no existe ningún paquete con el nombre "QL"`.
- **Causa Conceptual:** El código fuente invocaba de forma imperativa al macro de gestión de dependencias externas mediante la sintaxis prefija calificadora (`ql:quickload "local-time"`). Sin embargo, el entorno de ejecución de Common Lisp subyacente carecía del módulo Quicklisp inicializado en su espacio de nombres léxico global, provocando que el lector (*reader*) fallara en tiempo de compilación al no reconocer el prefijo del paquete como un símbolo válido del sistema.
- **Diagnóstico y Corrección:** Se identificó un acoplamiento innecesario con librerías externas para la funcionalidad del núcleo matemático del sistema. Como diagnóstico inmediato para aislar el entorno puro, se procedió a la remoción de la directiva de carga `ql:quickload` y a la refactorización de las

funciones de auditoría (registrar-cambio) para operar de forma nativa e inmutable con tipos de datos enteros crudos (marcas de tiempo Unix). Para entornos de producción que requieran formato de calendario, se determinó como corrección inyectar de manera explícita la carga del script de configuración del entorno mediante (load "~/quicklisp/setup.lisp") previo a la importación del dominio semafórico.

```
*** - READ en
      #<INPUT CONCATENATED-STREAM #<INPUT STRING-INPUT-STREAM>
      #<IO TWO-WAY-STREAM #<INPUT UNBUFFERED FILE-STREAM CHARACTER @122> #<OUTPUT UNBUFFERED FILE-STREAM CHARACTER>>>
      : no existe ningun paquete con el nombre "QL"
Es posible continuar en los siguientes puntos:
ABORT      :R1      Abort debug loop
ABORT      :R2      Abort debug loop
ABORT      :R3      Abort debug loop
ABORT      :R4      Abort debug loop
ABORT      :R5      Abort debug loop
ABORT      :R6      Abort debug loop
```

Fase 2: Autonomía y Ecosistema

La Fase 2 se centra en dotar al núcleo funcional de autonomía y persistencia. Se integró el gestor de paquetes Quicklisp para gestionar dependencias de forma profesional, y la librería local-time para abstraer el manejo de fechas, sustituyendo la complejidad de los enteros Unix (Epoch) por objetos cronológicos manipulables.

Guía de Ejecución y Pruebas:

Para probar el sistema se utilizó el REPL interactivo de SBCL dentro de Visual Studio Code.

1. Asegurarse de tener instalado SBCL y Quicklisp.
2. Abrir la terminal en la carpeta del proyecto y ejecutar sbcl.
3. Cargar el código con: (load "fase2.lisp")

Pruebas por Requerimiento:

Auditoría de Transiciones (Requerimiento 3 Modificado)

Para verificar que el sistema muestra fechas legibles en lugar de segundos crudos, ejecutar:

(registrar-cambio 1786044000 'en-rojo 'en-verde)

Resultado esperado:

Tiempo [2026-06-06 16:20:00]: la luz ha cambiado de <EN-ROJO> a <EN-VERDE>

Persistencia de Datos (Extensión 2):

Para demostrar que el sistema genera el archivo de reporte sin usar bucles imperativos, se implementó recursividad de cola:

```
lisp
(informe '(((1786044000 en-rojo en-verde)
            (1786044120 en-verde en-amarillo))))
```

Resultado esperado: el REPL devuelve ESCRITURA-COMPLETADA y se crea el archivo informe-ejecucion-semaforo.txt en la misma carpeta del proyecto.

Prueba 3 — Simulación Completa (Requerimiento 7)

Para simular un rango horario específico (ejemplo: 7 de junio de 2026, de 11:00 a 15:20):

```
lisp
(let* ((inicio (local-time:encode-timestamp 0 0 0 11 7 6 2026))
      (fin (local-time:encode-timestamp 0 0 20 15 7 6 2026))
      (datos (simular-linea-tiempo
                (local-time:timestamp-to-unix inicio)
                (local-time:timestamp-to-unix fin)
                'en-rojo)))
  (informe datos))
```

Este bloque convierte fechas legibles a formato Unix, ejecuta la simulación segundo a segundo mediante recursividad de cola y genera el archivo final con el historial completo de transiciones.

Captura 1: transiciones (normal y error):

Demostrar que la función evalúa correctamente los estados válidos y rechaza los inválidos.

```
* ;; 1. Camino Normal
(transicion 'en-rojo 'verde)
(EN-ROJO "cambiar-a-verde")
*

;; 2. Camino Normal (Intermitencia)
(transicion 'en-amarillo 'amarillo-intermitente)
(EN-AMARILLO "cambiar-a-amarillo-intermitente")
*

;; 3. Camino Alternativo / Error (Transición no permitida)
(transicion 'en-rojo 'amarillo)
(EN-ROJO ACCION-POR-DEFECTO)
```

Captura 2: Temporizador y Cálculos

Mostrar las funciones matemáticas y de planificación temporal.

```
;; 1. Temporizador (Ejemplo en el segundo 150 del ciclo)
(timer 150)
EN-VERDE
*

;; 2. Recomendación del ciclo actual (219 segundos)
(recomendacion-ciclo 219)
"Ciclo ineficiente (Peligro): Mayor a 150s, los usuarios tienden a evitarlo."
*

;; 3. Planificación (Ciclos en 60 minutos)
(ciclos-por-tiempo 60)
16
96
*

;; 4. Porcentajes de 1 hora
(informe-distribucion-1hora)
((EN-ROJO . 41.09589) (EN-VERDE . 54.79452) (EN-AMARILLO . 2.739726)
 (AMARILLO-INTERMITENTE . 1.369863))
*
```

Captura 3: Simulación en Consola

Demostrará la recursividad y cómo se imprime en pantalla usando la librería local-time.

```
;; Simulación corta para forzar el cambio de Rojo a Verde (segundo 88 al 91)
(defparameter *mis-datos* (simular-linea-tiempo 88 91 'en-rojo))
Tiempo [1970-01-01 00:01:30]: la luz ha cambiado de <EN-ROJO> a <EN-VERDE>
*MIS-DATOS*
*

;; Imprimir los datos acumulados en memoria
(print *mis-datos*)

((90 EN-ROJO EN-VERDE))
((90 EN-ROJO EN-VERDE))
*
```

Captura 4: Persistencia en Archivo

Ejecuta el informe con los datos generados en el paso anterior.

```
(defparameter *mis-datos* (simular-linea-tiempo 88 91 'en-rojo))
Tiempo [1970-01-01 00:01:30]: la luz ha cambiado de <EN-ROJO> a <EN-VERDE>
*MIS-DATOS*
*

(informe *mis-datos*)
NIL
* |
```

fase22.lisp

informe-ejecucion-semaforo.txt X

informe-ejecucion-semaforo.txt

```
1 Informe de Ejecución del Sistema Semafórico
2 =====
3 1970-01-01 00:01:30 - Transición: EN-ROJO -> EN-VERDE
4 =====
5 --- Fin del Informe ---
6
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

REPL

```
(defparameter *mis-datos* (simular-linea-tiempo 88 91 'en-rojo))
Tiempo [1970-01-01 00:01:30]: la luz ha cambiado de <EN-ROJO> a <EN-VERDE>
*MIS-DATOS*
*

(informe *mis-datos*)
NIL
* (informe *mis-datos*)|
```


Justificación del Paradigma

¿Por qué local-time?

Porque evita el acoplamiento de lógica temporal. Realizar cálculos de tiempo manualmente haría el código propenso a errores como años bisiestos y zonas horarias. Local-time actúa como un tipo de dato abstracto que garantiza exactitud cronológica.

¿Cómo se garantiza cero bucles imperativos?

La restricción de la cátedra prohíbe `do` y `loop`. La función procesar-registros fue implementada usando recursividad de cola pura, cumpliendo con la restricción sin sacrificar funcionalidad.

Bitácora de Depuración – Bug #1

Tipo de Error: Problema de evaluación de símbolos (UNBOUND-VARIABLE).

Contexto del fallo: Ocurrió durante la Fase 2 (Persistencia de Datos), al intentar pasar por argumento el historial de transiciones a la función informe mediante la instrucción (informe mis-datos).

Descripción y Causa Conceptual: El entorno de ejecución interrumpió el proceso invocando al depurador por un error de variable no enlazada (unbound variable). El error conceptual radicó en la evaluación de símbolos propios de Lisp. La variable había sido declarada previamente como `*mis-datos*` utilizando `defparameter`. A diferencia de otros lenguajes imperativos donde los caracteres especiales pueden ser meros decoradores, en Lisp los asteriscos (conocidos por convención como earmuffs para variables globales) forman parte literal y estricta del nombre del símbolo en el espacio de memoria. Al omitirlos, el evaluador intentó buscar un símbolo llamado exactamente `mis-datos`, el cual no existía en el entorno léxico ni global.

Solución Aplicada: Se interrumpió la sesión del depurador seleccionando la opción de reinicio [ABORT] para retornar al nivel principal (top level). Posteriormente, se corrigió la sintaxis invocando a la función con el nombre exacto del símbolo: (informe `*mis-datos*`), lo que permitió la correcta ejecución del efecto secundario (I/O).

```

*
(defparameter *mis-datos* (simular-linea-tiempo 88 91 'en-rojo))
Tiempo [1970-01-01 00:01:30]: la luz ha cambiado de <EN-ROJO> a <EN-VERDE>
*MIS-DATOS*
* (informe mis-datos)

debugger invoked on a UNBOUND-VARIABLE @10006F5CA0 in thread
#<THREAD tid=9872 "main thread" RUNNING {1104028083}>:
  The variable MIS-DATOS is unbound.

Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):
  0: [CONTINUE   ] Retry using MIS-DATOS.
  1: [USE-VALUE  ] Use specified value.
  2: [STORE-VALUE] Set specified value and use it.
  3: [ABORT      ] Exit debugger, returning to top level.

(SB-INT:SIMPLE-EVAL-IN-LEXENV MIS-DATOS #<NULL-LEXENV>)
0] 3

```

Bug #2: Conflicto con palabras reservadas del sistema (PACKAGE-LOCK-VIOLATION)

Síntoma: Al intentar cargar nuestro código final de la Fase 2 ejecutando (load “fase2.lisp”) en la terminal, la ejecución se cortaba abruptamente. El entorno nos abría el depurador con un error gigante que decía PACKAGE-LOCK-VIOLATION #<PACKAGE “SB-EXT”> :SYMBOL TIMER.

Causa Conceptual: Al principio, como novatos en el lenguaje, pensamos que era un error del Visual Studio Code o de la extensión que estábamos usando para Lisp, porque la sintaxis de nuestra función estaba perfecta. Sin embargo, el problema venía directo del compilador (SBCL). Resulta que Lisp organiza sus herramientas en “paquetes”, y el compilador SBCL ya tiene una función interna súper importante llamada timer (dentro de su paquete SB-EXT).

Para evitar que los programadores rompan el sistema sin querer, SBCL le pone un “candado” a sus funciones propias. Cuando nosotros escribimos (defun timer ...), Lisp pensó que estábamos intentando sobrescribir su herramienta interna y saltó a defenderse bloqueando la compilación.

Diagnóstico y Corrección: Nuestra primera “solución rápida” había sido poner un 2 en la terminal para seleccionar la opción [UNLOCK-PACKAGE] que nos ofrecía el depurador. Básicamente, estábamos forzando la cerradura. Al investigar más, nos dimos cuenta de que esto es una pésima práctica de programación.

La solución definitiva y correcta fue mucho más simple: dejamos de pelear con las palabras reservadas del sistema y renombramos nuestra función. Cambiamos el nombre de timer a temporizador tanto en su definición como en la parte donde la llamábamos dentro de simular-linea-tiempo. Al hacer este pequeño cambio, el archivo cargó limpio a la primera, devolviendo un hermoso T en la consola sin interrupciones.

Ejemplo:

```
Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):
  0: [CONTINUE      ] Ignore the package lock.
  1: [IGNORE-ALL    ] Ignore all package locks in the context of this operation.
  2: [UNLOCK-PACKAGE] Unlock the package.
  3: [RETRY         ] Retry EVAL of current toplevel form.
  4:                Ignore error and continue loading file "C:\\Users\\enzo\\Videos\\fase22.lisp".
  5: [ABORT         ] Abort loading file "C:\\Users\\enzo\\Videos\\fase22.lisp".
  6:                Exit debugger, returning to top level.

(PACKAGE-LOCK-VIOLATION #<PACKAGE "SB-EXT"> :SYMBOL TIMER :FORMAT-CONTROL "proclaiming ~S as a function" :FORMAT-ARGUMENTS
(TIMER))
0] 2

; file: C:/Users/enzo/Videos/fase22.lisp
; in: DEFUN CICLOS-POR-TIEMPO
;   (DURACION-CYCLE-AUXILIAR)
;
; caught STYLE-WARNING:
;   undefined function: COMMON-LISP-USER::DURACION-CYCLE-AUXILIAR
;
; compilation unit finished
; Undefined function:
;   DURACION-CYCLE-AUXILIAR
; caught 1 STYLE-WARNING condition
```

Fase 3: Estudio Comparativo

Introducción:

Scala (Scalable Language) es un lenguaje de programación de propósito general que combina de forma nativa el paradigma orientado a objetos con el paradigma funcional. Fue creado por Martin Odersky y lanzado en 2004. Corre sobre la Java Virtual Machine (JVM), lo que le permite una interoperabilidad total con el ecosistema Java y una adopción directa en entornos empresariales que ya lo utilizaban.

Scala permite escribir código tanto en estilo imperativo como en estilo puramente funcional, siendo esta dualidad una de sus características más destacadas y también uno de los puntos de análisis de este trabajo.

Industrias y Áreas de Uso:

Scala tiene presencia fuerte en áreas donde se requiere procesamiento de datos a gran escala, sistemas distribuidos y backends de alta concurrencia. Entre los sectores donde más se utiliza se encuentran:

- Sistemas Financieros
- Telecomunicaciones
- Streaming de Datos

Empresas que utilizan Scala:

- Twitter / X
- LinkedIn
- Netflix
- Airbnb
- Spotify
- Apple

Implementación de Funciones:

Transición:

```
// =====
// FUNCIÓN: transicion
// NATURALEZA: Pura (Determinista, sin efectos secundarios en el entorno)
// ESTRATEGIA DE CONTROL: Evaluación Condicional Estructurada
// IMPACTO EN MEMORIA: No Destructiva (Retorna una nueva tupla inmutable)
// =====

def transicion (colorActual: String, cambiarA: String):
  (String, String) = {if (colorActual == "en-rojo" && cambiarA == "verde") {("en-rojo", "cambiar-a-
verde")}}

  else if (colorActual == "en-verde" && cambiarA == "amarillo")
  { ("en-verde", "cambiar-a-amarillo")}

  else if (colorActual == "en-amarillo" && cambiarA == "amarillo-intermitente")
  { ("en-amarillo", "cambiar-a-amarillo-intermitente") }

  else if (colorActual == "amarillo-intermitente" && cambiarA == "rojo")
  { ("amarillo-intermitente", "cambiar-a-rojo") }
```

```

else {
    (colorActual, "accion-por-defecto")
}

```

Timer:

```

// =====
// FUNCIÓN: timer
// NATURALEZA: Pura (Dado un timestamp idéntico, siempre retorna el mismo color)
// ESTRATEGIA DE CONTROL: Evaluación Condicional por Rangos Temporales
// IMPACTO EN MEMORIA: No Destructiva (Asigna valores inmutables en Stack) //
=====

```

```

def timer (timestampActual: Long):
    String = {val duracionTotal = 219 // 90 + 120 + 6 + 3
    val momentoCiclo = timestampActual % duracionTotal
    if (momentoCiclo < 90) {"en-rojo"}
    } else if (momentoCiclo < 210) {"en-verde" // 90 + 120
    } else if (momentoCiclo < 216) {"en-amarillo" // 90 + 120 + 6
    } else {"amarillo-intermitente"}
    }

```

Conclusión Grupal:

Analizar la lógica de un semáforo de Lisp a Scala fue una experiencia desafiante para los cuatro. Uno de los puntos que más debatimos en el grupo fue cómo representar los "símbolos" de Lisp en Scala, lo que nos llevó a experimentar con cadenas y a lidiar con errores típicos de comparación. Resolver estas fallas nos hizo comprender que Scala simplifica la legibilidad del código frente a Lisp, pero exige un control mucho más estricto de las estructuras de datos.

Preguntas:

1. **Scala combina la Programación Orientada a Objetos (POO) con la Funcional. ¿Cómo estructuraron el semáforo? ¿Usaron una clase**

tradicional, un `object` (Singleton), o `case classes`? Justifiquen desde el diseño funcional.

Estructuramos el sistema del semáforo declarando las funciones transición y dentro de un object (**objeto Singleton**) en Scala, utilizando tipos de datos primitivos e inmutables (`string` para los estados/acciones y `long` para el timestamp)

Justificación desde el diseño funcional:

Descartamos el uso de una clase tradicional porque no necesitamos crear múltiples instancias del semáforo, ni requeríamos que el programa retenga un estado interno que mute todo el tiempo

Uso de funciones puras: cuando agrupamos dentro de un object, tratamos a transición y timer como funciones puras y deterministas. Esto significa que no provocan efectos secundarios, reciben sus datos de entrada, realizan el cálculo lógico y retorna un nuevo valor inmutable, sin alterar absolutamente nada del entorno.

Datos seguros: en el caso de transición, cuando no se cumple ninguna condición, el sistema responde devolviendo (`colorActual`, “acción-por-defecto”). Esto demuestra un diseño puramente funcional donde los datos viejos no modifican, sino que se genera una nueva estructura inmutable en memoria.

2. Comparen la manipulación de listas en Scala (métodos como `.map` o `.filter`) contra las funciones de orden superior de Common Lisp. ¿Cuál resulta más legible y por qué?

Haciendo la comparación de las manipulaciones de las listas, tanto en common Lisp y en scala, nos resulta más legible y fácil Scala por las siguientes razones:

El orden de lectura: en scala usas la notación de puntos y encadenas los métodos uno atrás del otro, esto hace que el código se pueda leer de corrido en el mismo orden en que se ejecutan las cosas, en cambio en common Lisp Tienes una sintaxis prefija que obliga a leer de adentro hacia afuera y de derecha a izquierda

Menos utilización de paréntesis: scala usa llaves y paréntesis normales para los argumentos. Lisp utiliza muchos paréntesis haciendo que se acumulen todos al final de la línea; haciendo que tengas que pasar ratos revisando los paréntesis además que es difícil encontrar un error a simple vista.

